

FORMATION EMBARQUÉE

Guide de la formation

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

code/ — les corrigés en fichiers sources réels

Tous les corrigés des TD sous forme de projets **compilables et testés**. Chaque dossier a son **Makefile** (quand l'outil s'y prête) : **make test** compile puis exécute les tests — c'est la façon dont ce code a été validé.

Dossier	Contenu	Comment vérifier
<code>c/</code>	TD 01 : <code>bits</code> , <code>ring_buffer</code> , <code>fsm_feu</code> , <code>trame</code> + tests unitaires	<code>make test</code> (gcc, <code>-Wall -Wextra -Werror</code>)
<code>cpp/</code>	TD 02 : <code>TamponCirculaire<T,N></code> , <code>IAfficheur</code> + mock LCD, <code>ChipSelect</code> RAIL, <code>FeuTricolore</code> + tests	<code>make test</code> (g++ C++17)
<code>java/</code>	TD 05 : hiérarchie <code>Capteur / Station</code> , <code>DecodeurTrame</code> (piège du byte signé), <code>ProdCons</code>	<code>make test</code> puis <code>make prodcons</code> (JDK ≥ 17)
<code>vhdl/</code>	TD 04 + TP 2 : <code>mux4</code> , <code>compteur_bcd</code> , <code>debounce</code> , <code>uart_tx</code> , <code>uart_rx</code> complet , testbenchs dont la boucle exhaustive 256/256	<code>make test</code> (GHDL) ; <code>make ondes</code> pour GTKWave
<code>arduino/</code>	TD 03 : <code>chenillard/</code> et <code>thermostat/</code> (sketches complets)	ouvrir dans l'IDE Arduino ou sur wokwi.com
<code>python/</code>	TP 4 : <code>supervision.py</code> , client Modbus TCP complet (mot de vie, journal CSV, commandes)	<code>pip install pymodbus</code> puis <code>python3 supervision.py <ip></code>
<code>st/</code>	TD 07/08 : <code>moyenne_glissante.scl</code> , <code>porte_garage.scl</code> , <code>alternance_pompes.st</code>	à coller dans un FB TIA Portal / Control Expert

Comment travailler avec ces corrigés

1. **Fais d'abord l'exercice toi-même** à partir de l'énoncé du TD.
2. Compare ensuite avec le fichier source ici : lis les commentaires, ils expliquent les choix (et les pièges) autant que le TD.
3. **Casse les tests** : modifie une ligne du corrigé (enlève un `volatile` , inverse une condition) et regarde quel test échoue — c'est le meilleur moyen de comprendre à quoi chaque ligne sert.
4. Réutilise : `ring_buffer` et `TamponCirculaire` resservent tels quels dans les TP 5 (console STM32) ; `uart_tx/rx` sont le cœur du TP 2.

État de validation

- `c/` , `cpp/` , `java/` : compilés et tests exécutés (zéro warning avec `-Werror`).
- `vhdl/` : les trois testbenchs passent sous GHDL, dont `tb_uart_boucle` — **256/256 octets** transmis/reçus sans erreur.
- `python/` : nécessite un automate (ou simulateur) Modbus pour tourner ; la syntaxe est vérifiée.
- `arduino/` , `st/` : à compiler dans leurs IDE respectifs (IDE Arduino / Wokwi, TIA Portal / Control Expert).